

Red Hat Certified Specialist in Containers (EX188)

- Lesson 1 : Why use Containers
- Lesson 2 : Working with **podman**
- Lesson 3 : Container Images
- Lesson 4 : Creating Custom Container Images
- Lesson 5 : Configure Persistent Storage
- Lesson 6 : Troubleshooting Containers
- Lesson 7 : Integrating Podman Containers with Systemd
- Lesson 8 : Microservices and Multi-container applications
- Lesson 9 : An introduction to Container Orchestration.
- Lesson 10 : Sample Exam

Learning Objectives:

1.1 What are Containers?

- A container is way to start an application in a fully isolated environment from an image that contains all the application dependencies.
- Running containers is about starting applications in an efficient way.
- Container images are standardized by the Open Containers initiative (OCI), which allows you to run containers on top of any compatible container engine, including Podman, Docker, and Kubernetes.
- As the container image has all the required dependencies included, it doesn't depend on anything on the host operating system apart from the container runtime.
- Containers are Linux: the Linux kernel provides features that allow for full isolation and resource restrictions.

1.2 Running Applications for an Operating System.

- Non-containerized applications need to be installed.
- For each operating system version, a specific installer needs to be provided.
- To provide required libraries, dependencies need to be provided.
- Only one version of a specific dependency can be installed at the same time.
- Updating applications necessarily involves bringing down the application.
- A specific application version needs to be developed for each operating system it should be installed on.

1.3 Running Applications as Containers

- The container image runs the application in an isolated environment.
- The image runs on any operating system, if the CPU platform is the same.
- An application developed for ARM doesn't work on Intel architecture.
- The container images have everything that is needed to start the application.
- As it runs in an isolated environment, its dependencies are independent of dependencies used by other applications that run on the same host.

1.4 Containers versus Virtual Machines

- A virtual machine runs a complete operating system on top of a hypervisor.
- Each virtual machine requires a full operating system installation.
- This does allow for running multiple different operating systems on top of hypervisor.
- It also is resource inefficient, as each virtual machine needs to load a full environment.
- Also, standardization of virtual machines is low: a VM that was created to run on one hypervisor usually needs to be converted to run on another hypervisor.
- A container is just the application with all its dependencies, provided as an image that runs on top of the container engine.
- Containers share the operating system of the container host, which makes them more efficient.
- This does reduce options to run containers on different operating systems.
- It also makes containers very efficient.
- Also, as container images are standardized through the Open Containers Initiative (OCI), they easily run on different container engines.

1.5 Why Podman Beats Docker

- Container technology started in the 1980's with chrooted applications.
- The container revolution started when Docker added a daemon and a container registry to Linux native containers in LXC.
- Podman was introduced later and provides improvements to Docker.
- Red Hat replaced Docker with Podman in RHEL 8.
- As Docker and Podman run the same OCI images, replacing Docker with Podman was not a big issue.
- Also, the **podman** command was developed to be fully compatible with the docker command.
- Podman uses the fork/exec model, where Docker uses a daemon.
- Podman is rootless by nature, where Docker uses elevated privileges by default.
- Podman supports running containers in separate user namespaces, which offers true isolation between processes.
- Podman fully integrates with system, which allows for starting Podman containers as system services.
- Podman can run Kubernetes Pods and fully supports the YAML format.
- When containers are running, Podman is not required to remain running, which allows for easy upgrades.

Lesson 2: Working with podman

- 2.1 Registry Access
- 2.2 Running Containers

- 2.3 Rootless versus Root Containers
- 2.4 Managing Containers
- 2.5 Container Networking
- 2.6 Accessing Containers
- 2.7 Restricting Containers

2.1 Registry Access

- The most essential part of any **podman** command is the container image name.
- Images are fetched from registries.
- Podman doesn't work with a default registry, which is why the image name should include the full name as it occurs on the registry.
- This is known as the Fully Qualified Container Name (FQCN).
- So instead of **podman run -d nginx**, you should use **podman run -d docker.io/library/nginx**
- Once the image is locally stored, short names can be used.
- Many registries require authentication to get full access to their images.
- For Red Hat registry access, you can use your Red Hat Developer account (developers.redhat.com).
- Docker registry access is free but comes with a pull rate limitation.
- To get less restricted access to the Docker registry, create a free Docker account, or purchase a Docker professional account.
- Use **podman login** to authenticate on any registry.
- **podman search ubi9**
- **podman pull registry.redhat.io/ubi9/ubi**
- **podman login registry.redhat.io**
- **podman pull registry.redhat.io/ubi9/ubi**

Red Hat Registry Access Command

#podman search ubi 9	-	To search ubi9 image Red Hat Registry.
#podman pull registry.redhat.io/ubi9/ubi	-	Try pull image from registry (see error)
#podman login registry.redhat.io	-	To login in Red Hat Registry then type username and password.
#podman pull registry.redhat.io/ubi9/ubi	-	To pull images successfully.

2.2 Running Containers

- The default command that is started by a container image is hard coded in the container images.
- When starting the container, the default command is executed and when this command exits, the container stops.
- To run a different command while starting the container, the command can be added as a container argument.
 - Use **podman run -it** if the command requires access to an interactive shell.
- To run a separate command on a container that is already running, use **podman exec**.

Commands Overview

#podman login	-	Is used to log in on a specific registry.
#podman search	-	Searches for container images.
#podman pull	-	Fetches images from the registry and stores locally.
#podman images	-	Shows images stored by the current user.
#podman ps	-	Shows contains running for the current user.
#podman ps -a	-	Also shows contains no longer running.
#podman run	-	pull the container images and runs it. Once the main application stops, the container stops.

Demo: Running Containers

```
#podman search uni
```

```
#podman pull registry.access.redhat.com/ubi8/ubi
```

```
#podman imaes
```

```
#podman run ubi
```

```
#podman ps
```

```
#podman ps -a
```

```
#podman run ubi echo hello word
```

```
#podman run nginx
```

```
#podman run -d nginx
```

```
#podman run -it ubi /bin/bash
```

2.3 Rootless versus Root Containers

- Podman containers are started by a specific user in a user specific namespace.
- The user namespace provides isolation for the container.
- As a result, containers started by one user are not visible by other users.
- Also, if a container runs with root privileges, these privileges don't get beyond the current user namespace.
- Running containers in user namespaces makes them secure by default.
- A rootless container as non-privileged user.
- Any user starts rootless containers.,
- After starting the container as a non-root user, within the container any UID (including root) can be used as it is a namespace UID that exists in the container only.
- This container internal user is mapped to a user on the host operating system for file access.
- A root container runs with root privileges and for that reason it is dangerous.
- Rootless containers do not get an IP address.
- Rootless containers cannot bind to a privileged port.
- Rootless containers have limited access to the filesystem.

Demo: Running root[less] Containers

```
#podman run -d docker.io/library/nginx
```

```
#ip a
```

```
#podman inspect -l -f "{{.NetworkSettings.IPAddress}}"
```

```
#podman run -d nginx
```

```
#ip a
```

```
#podman inspect -l -f "{{.NetworkSetting.IPAddress}}"
```

Note that the **-l** flag refers to the last container that was started and takes away the need to specify a name.

Containers and the Host OS

- From the host OS perspective, a container is just a process.

- A well-restricted process though, such as namespaces, cgroups, and SELinux limit what the container can do.
- Use **ps faux | less** and look for processes that are started by **canmon**.
- The Linux kernel ensures that these processes are restricted regardless of resources usage as well as access to other resources.

#ps faux | less

2.4 Managing Containers

Common Container Management

- #podman ps -a - shows current and past running containers.
- #podman stop - stops a running container.
- #podman start - Starts a stopped container again.
- #podman inspect - shows properties used by containers and images.
- #podman exec -it containername sh - runs a shell as the primary process in a container (and doesn't start the default command).
- #Ctrl-p, Ctrl-q - detaches from a current interactive session with a container.
- #podman attach containername - connects to a current interactive session on a container.

Demo: Managing Containers

- #podman run --name myweb -it nginx sh
- #ps aux
- #Ctrl-p, Ctrl-q
- #podman ps
- #podman inspect myweb
- #podman image inspect nginx
- #podman attach myweb
- #exit
- #podman ps -a

Using Variables

- Container images normally don't contain site-specific information.
- To provide site-specific information, environment variables can be provided.
- The most straightforward way to provide these variables is by starting the container with the **-e key=value** option.
- Some images refuse to start if the right variables are not provided.
- To analyze if a container needs additional information while starting, use the **podman logs containername** command.

Demo: Running Mariadb

```
#podman run dokcer.io/library/mariadb
```

```
#podman ps -a
```

```
#podman logs <containername>
```

```
#podman run -d -e MYSQL_ROOT_PASSWORD=password mariadb
```

```
#podman run -d -e MARIADB_ROOT_PASSWORD=password mariadb
```

```
#podman ps
```

2.5 Container Networking

- Podman containers, by default, connect to the podman network.
- Use **podman network** to see current networks.
- To start containers in a different network namespace, use **podman network create** to create additional networks.
- Traffic between networks is blocked.

2.6 Accessing Containers

- Containers cannot be reached directly by external users.
- To access a container, a port can be exposed in the host that runs the container.
- Ports can only be exposed while starting the container, and not for currently running containers.
- Rootless containers cannot be exposed on privileged ports.
- In port forwarding, a source IP address can be specified to allow access only if traffic comes from a specific IP address: **podman run -d -p 127.0.0.1:8080:80 nginx**

Demo: Exposing Containers

- ```
#podman run -d -p 8080:80 nginx
```
- ```
#curl localhost:8080
```
- ```
#podman ps
```
- ```
#podman port
```

2.7 Restricting Containers

- On the Linux host, cgroups is used to implement resource restrictions.
- This allows the Linux kernel to optimize the use of the available CPU, memory, and disk I/O resources.
- To run a container with resource restrictions, specific arguments can be use while running the container.
- **sudo podman run --memory=1G --cpu-shares=512 -d bitnami/nginx:latest**
- See #man podman-run for more details

Enabling Restrictions for Rootless Containers

- By default, cgroup restrictions don't work for rootless containers.
- To change this:
 - Create the file `/etc/containers/containers.conf`:
[containers]
Pids_limit=0

#mkdir /etc/system/system/user@.service.d/
Create the file `/etc/system/system/user@.service.d/delegate.conf`
[Service]
Delegate=memory pids cpu io

Demo: Running Restricted Containers

```
#vim /etc/containers/containers.conf  
[containers]  
pid_limit=0
```

```
#mkdir /etc/system/system/user@.service.d
```

```
#vim /etc/system/system/user@.service.d/delefate.conf  
[Service]  
Delegate=memory pids cpu io
```

```
#systemctl daemon-reload
```

```
#podman run -d -m 512M registry.redhat.io/ubi9/ubi sleep 1800
```

```
#podman stats
```

- Use the appropriate command to find the ubi8 image.
- Download the image without running it.
- Inspect the image to find out what is used as the default command.
- Run the image with an interactive shell and find which operating system and which kernel it uses.

Lesson 3: Container Images

Learning objectives:

- 3.1 Using Image Registries
- 3.2 Pulling Images
- 3.3 managing Images
- 3.4 Creating Custom Images from Running Containers
- 3.5 Using Tags

Lesson 3 Lab: Managing Images

3.1 Using Image Registries

- Container images are provided through image registries.
- In Podman, there is no default image registry.
- To indicate where to fetch the image, use fully qualified image names: `docker.io/library/nginx` and not just `nginx`.
- However, a list of unqualified search registries can be provided through `/etc/containers/registries.conf`.
- While referring to images, Podman will search through these registries and if needed, presents a list of all images that were found.
- As discussed before, many registries require authentication to access images.

Using Common Image Registries:

- `docker.io` is the biggest image registry; it contains many images that are offered in the Open Containers initiative (OCI) format.
- `registry.access.redhat.com` is a publicly available Red Hat-hosted image registry that contains images optimized for Podman and Red Hat OpenShift.
- `Registry.redhat.io` provides images for access by registered Red Hat users only. A free developer's account qualifies for accessing this registry.
- `quay.io` is a community image registry sponsored by Red Hat.
- Many other public registries are available.
- Alternatively, private registries can be used.

Demo: Using Image Registries

- `#podman run -d nginx`
- `podman images`
- `podman run docker.io/library/busybox`

3.2 Container Images

- Images are composed of different storage layers.
- Working with layers allows developers to use standard images and only add modifications on top of them.
- While fetching images, all layers are downloaded as separate processes and combined into a runnable image.
- If updates are available at any layer, only the specific layer needs to be updated and not the entire image.

Pulling Images:

- While running containers, the image is fetched (“pulled”) before the container is started.
- To prefetch images, you can use the **podman pull** command.
- The default pull policy will only fetch the image if it’s not yet locally available.
- To overwrite the default pull policy, use **podman run --pull=option imagename**, where options is never, always, newer or missing.
- Locally stored images can be used by their name or by the image ID.

Demo: Pulling Images

- `#podman run --pull=never -d docker.io/library/mariadb`
- `#podman run --pull=newer -d dokcer.io/library/mariadb`
- `#podman image list`
- `#podman image inspect mariadb`

Investigating Images:

- Efficient images are composed of a minimal amount of image layers.
- Use **podman image tree imagename:latest** to show image layers.

`#podman image tree busybox:latest` - To show image layers

`#podman image tree mariadb` - to show image layers.

3.3 Managing Images:

- Images are stored separately for each user.
- The **podman image** command provides different options for managing images.

`#podman image list` - Lists available images.

`#podman image inspect` - Shows image details.

`#podman image rm` - Removes specific images.

`#podman image prune` - Removes unused images.

3.4 Creating Custom Images from Running Containers:

- Custom images can be created in different ways:
 - Build from scratch using Containerfile.
 - Create from changes made to a running container.
 - Create by using advanced tools such as **buildah**.
- To save changes that are made inside a running container to an image, use **podman commit**.

Demo: Using podman commit

```
#podman run -it --name customweb nginx sh - To create a container and go inside.
```

```
    #touch /tmp/testfile - inside container
```

```
    #exit - To exit from container.
```

```
#podman commit customweb nginx:custom - To creating image using commit command.
```

```
#podman images - To see available images.
```

```
#podman run -it nginx:custom ls -l /tmp/testfile
```

(To create container using custom image then go inside and check created file available or not.)

3.5 Using Tags:

- A tag is a label that can be used to identify a container image.
- Common tags include version information, or the word “latest” to refer to the most recent image version that is available.
- Additional tags can be set to images as identifier labels.
- Each image has a tag, and the **podman image tag** command can be used to set tags on images.

```
#podman image tag docker.io/library/nginx myimage:1.0
```

(To tag the image, tag name is myimage:1.0)

```
#podman images - To see available images
```

Lesson 3 Lab: Managing Images

- Prefetch the nginx image from docker.io
- Run a container based on the prefetch nginx image and ensure that under no circumstances will a new image instance be pulled.
- Open a shell in the running container and create the file /tmp/labfile, using **touch /tmp/labfile**

- Save the changes made inside the container to new container image with the name localhost/nginx:local
- Run an interactive shell on a container started from the image localhost/nginx:local and verify that /tmp/labfile exists.

```
#podman search nginx | grep docker      -      To search nginx image.

#podman pull docker.io/library/nginx    -      To pull image

#podman run --pull=never nginx

#podman run --pull=never -it nginx

#podman run --pull=never -it nginx sh    -      Run and go inside the container.

        #touch /tmp/labfile      -      To create a file inside the container.
        #exit                    -      To exit from the container.

#podman ps -a                          -      To see start and stop container.

#podman commit 6d5 localhost/nginx:local -      To commit the container.

#podman images                         -      To list images

#podman run -it localhost/nginx:local sh -      To run container and go inside.

        #ls -l /tmp/labfile      -      To see this file inside the container
        #exit                    -      To exit from the container

#podman run -it localhost/nginx:local ls -l /tmp
```

(To run container and see under /tmp directory created file available or not inside this running container.)

Lesson 4: Creating Custom Container Images

Learning Objects

- 4.1 Using Containerfile
- 4.2 Applying Best Practices to create a Containerfile.
- 4.3 Advanced Containerfile Options.
- 4.4 Containerfile ARG and ENV
- 4.5 Managing Container User IDs
- 4.6 Hosting a Private Registry

Lesson 4 Lab: Creating Custom Container Images

4.1 Using Containerfile

Understanding Containerfile:

- A Containerfile (AKA Dockerfile) contains instructions for building custom container images.
- Each instruction causes a change that results in an image layer added to the final image.
- In a Containerfile, **FROM** is used to refer to base image.
- Different common base image exists
 - . alpine
 - . busybox
 - . UBI

Universal Base Image:

- Red Hat provides Universal Base Images (UBI) which exist in different versions.
 - Standard: is the standard image and contains most of the essential Linux utilities.
 - Init: simplifies running multiple applications on one container through system
 - Minimal: uses the **microdnf** package manager for increased efficiency.
 - Micro: the smallest, UBI, doesn't include a package manager
 - Red Hat also provides UBI images that include popular runtimes.

Using Containerfile:

- Developers can write their own Containerfile.
- Alternatively, Containerfiles can be provided.
- Use **podman build -t image:name .** to build the image:name from a Containerfile in the current directory.

Common Containerfile Instructions:

FROM: Identifies the base image to use.

WORKDIR: Sets a working directory for all next instructions.

COPY: Copies files from the host to the image.

ADD: Copies files from host or URL or unpacks files from tar archives.

RUN: Runs a command while building the image. Files created by using RUN exist in the final image created.

ENTRYPOINT: Defines the default command to use. Commonly set to a shell.

CMD: Specifies a command to run while starting the resulting image.

USER: Defines the user account that runs the container commands.

LABEL: Defines a key-value pair used for informational purposes.

EXPOSE: Shows which port is exposed by the container (without actually exposing it).

ARG: Defines arguments that can be used while building the image from the Containerfile.
ENV: defines environment variables for use in the container.
VOLUME: specifies where to store data outside of the container.

Demo: Using Containerfile

Create a Containerfile with the following content:

```
FROM registry.redhat.io/ubi9/ubi  
RUN dnf install nmap -y  
CMD echo hello world
```

Commented [BD1]: Example of Containerfile

- Create the image, using **podman build -t myapp .**
- Use **podman images** to verify its creation
- RUN it: **podman run myapp**
- Use **podman images tree --whatrequires registry.redhat.io/ubi9/ubi:latest**

4.2 Applying Best practices to Create a Containerfile:

- Use a base image that contains what you need and not much more.
- Consider defining a WORKDIR that will be used as the working environment while building the container image.
- Avoid using unnecessary RUN statements as each will add another layer to the container image.
- Use labels to provide more information about the images.

Demo: Optimizing a Containerfile:

- The course Git repository <https://github.com/sandervanvugt/ex188> has the subdirectory sample-containerfile. This containerfile has a few options for optimization.

#dnf install git -y - To install git.

#git clone <https://github.com/sandervanvugt/ex188> - To clone

4.3 Advanced Containerfile Options

Using Multistage Builds

- You should always try to keep images as small as possible.
- This isn't always obvious if, instance, you compile an application using RUN statements while building the images.
- One solution would be to clean up all builder tools after compiling your application.

- Another solution is the multistage build.
- In a multistage build you build the application in stage one. In stage two, you all access the compiled application from stage one image.
- In this scenario, the stage one image is not in the final image which helps to keep it minimal.
- See multistage/Containerfile in the Git repo for an example.

Demo: Using Multistage Builds:

- This demo requires access to the course git repository at <https://github.com/sandervanvugt/ex188>
- **#podman build -t multi ex188/multistage-container**
- **#podman image tree multi -** shows that the busybox image layer is not present.

Using Volume:

- Container storage by nature is ephemeral.
- Volumes can be used to provide persistent storage.
- Within the Containerfile, use **VOLUME /my/app** to create a directory in the image filesystem where volumes can be mounted.
- Using a **VOLUME** instruction in Containerfile does NOT actually create the volume itself, it indicates to users of the image that the image storage is meant to be
- After defining it in the Containerfile, you actually need to run a container that mounts a volume on the specified directory.

Demo: Using Volumes

- This demo requires access to the course Git repository
- **#podman build -t countdown ex188/countdown-containerfile**
- **#podman run -v countdown:/mydata countdown**
- **#podman volume list**
- **#podman volume inspect [volume name]**
- **#ls -l /var/lib/containers/storage/volumes/countdown/_data/**

CMD versus ENTRYPOINT

- Containers are used to start a command
- This command can be specified as **ENTRYPOINT** or as **CMD**
- A command that is specified as **CMD** in the image, can be overwritten to run something else
 - **#podman run -it nginx sh -** will start a shell
- If the command is specified as **ENTRYPOINT**, arguments provided while running the container will not overwrite the default command, but specify a different argument to the **ENTRYPOINT** command
 - **#podman run countdown 2**

- Also try: `#podman run -it countdown sh`

4.4 Containerfile ARG and ENV:

Understanding ARG and ENV

- While building container images, environment variables can be used.
- These variables must be declared in the Containerfile using ARG.
- The value assigned to the ARG variables can be set while defining the ARG in the Containerfile. Example: ARG VERSION="1.2"
- Alternatively, while building the Containerfile the `--build-args KEY=VALUE` option can be used to define available variables.
 - `#podman build --build-arg version="1.2" -t versionapp .`

Demo: Using ARG and ENV

```
FROM ubi9/ubi
ARG argusr
ENV user=${arguser}
RUN mkdir /etc && \ touch /test/${user}
CMD ls -l /test/
```

```
#podman build --build-arg arguser=anna -t argtest .
```

4.5 Managing Container User IDs:

Understanding Rootless Containers

- Podman containers can be started as a rootless container:
 - The containerized process does not use the host root user.
 - The root user inside the container is mapped to a limited user outside of the container.
 - The container runtime does not need to be started by the root user.
- When Podman containers are started by non-privileged users, they are rootless by default.
- If within the container the root account is used, this account is mapped to a non-root user on the host OS.
- It still is better to run non-root users within the containers as well.

Containerfile Users

- Many images by default run as root.
- In Containerfiles, if no user account is specified, the default image user account is used.
- After using `RUN useradd` within the Containerfile, this newly added user can be used as the standard user.

- This is better, because even a root user in a restricted environment that is started by a non-root user container could be a security risk.

Understanding User Mappings

- When non-root users start a container, the user within the container is mapped to a user on the host OS.
- Container user id 0 maps to the UID of the user that started the container.
- Other users in the container are mapped to ID+999999 on the host.
 - So a mysql user with UID 27 in the container would get UID 100026 on the host.
 - Required host users are dynamically created.
- The `/etc/subuid` and `/etc/subgid` files contain the name of the current user, the starting dynamic UID on the host (typically 100000), and the number of UID's available (typically 65536).
- Use **podman top <containerID> huser user** to verify mappings for currently running containers.

Demo: Mapping UIDs

- Create a Containerfile with the following content:
FROM registry.access.redhat.com/ubi9/ubi
CMD ["sleep", "3600"]

```
#podman build -t sleeper .
#podman run -d --name sleeper sleeper
#podman top sleeper huser user
```

- Add the following to the Containerfile
RUN useradd sleeper
USER sleeper

```
#podman build -t sleeper2
#podman run -d --name sleeper2 sleeper2
#podman top sleeper2 huser user
```

Rootless Container Limitations

- Some applications cannot easily run as a rootless container.
- Apache, for instance, starts a main process with root privileges, which next spawns child processes that run as a non-privileged user.
- For this and similar applications, Red Hat provides customized image that runs as a non-privileged user.
- Bitnami is also a common provider of rootless images.
- Rootless containers cannot bind to a privileged port.

- Also, from within a rootless container some utilities (like ping) cannot be used.

4.6 Hosting a Private Registry

Running Private Registries

- A private registry can be created for more convenient access to images.
- Also, it's more secure location for images that are for internal use only.
- To run a private registry, use the registry container.
- To upload images to a private registry, tag the image with the registry location:
#podman tag fedora:latest localhost:5000/myfedora:latest

Accessing Insecure Registries

- Public registries should use TLS certificates for security.
- For private registries, insecure access should be allowed.
- To configure a registry for insecure access, create a file **/etc/containers/registries.conf.d/myregistry.conf** with the following contents:

```
[[registry]]
location="localhost:500"
insecure=true
```

- Alternatively, use **#podman push --tls-verify=false**

Demo: Running a Private Registry

```
#systemctl enable podman-restart.service --now
```

```
#podman run -d -p 5000:5000 --restart=always --name registry
docker.io/library/registry
```

```
#podman pull fedora
```

```
#podman images
```

```
#podman tag fedora:latest localhost:5000/myfedora
```

```
#podman push localhost:5000/myfedora --tls-verify=false
```

```
#podman rmi fedora
```

```
#podman exec -it registry sh; find . -name="myfedora"
```

```
#podman pull localhost:5000/myfedora --tls-verify=false - Download it again
from your own local registry.
```

Lesson 4 Lab: Creating Custom Container Images

- Write a Containerfile that creates a container image with the name `countdown` that starts the countdown script, provided in the course Git repository. Make sure to meet the following requirements:
 - The image is based on `ubi9` image.
 - While building the image, the commands **`dnf install bash`** and **`dnf clean all`** need to be executed in the most efficient way.
 - The script is started with a default counter of 1 minute. If so required, users that run the container should be able to provide the number of minutes as an argument to the **`podman run`** command.
 - Run the registry container and configure it as an internal registry server.
 - Upload the container image you just created to this registry container.

`#mkdir lab` - To create a directory

`#vim Containerfile` - To create a Containerfile like Dockerfile

FROM registry.access.redhat.com/ubi9/ubi

COPY countdown .

ENTRYPOINT ["/countdown"]

CMD ["1"]

`:wq` - To save and quit

`#podman build -t lab4 .` - To create an image using Containerfile.

`#podman images` - To list image.

Lesson5: Configuring Persistent Storage

Learning Objectives

5.1 Overlay Union Filesystem

5.2 Using Persistent Host Storage

5.3 Managing Permissions

5.4 Managing SELinux Settings

5.5 Configuring Storage for Stateful Applications

5.1 Overlay Union Filesystem

- Container images are layered: multiple layers are used to store specific parts efficiently.
- While building an image, each instruction adds a layer to the container.
- As previous layers cannot be modified, this should be carefully considered.
- This is also why only one RUN statement should be used in Containerfile.
- To investigate different layers, use **podman image tree <imagename>**
- Modifications made by users are written to a small ephemeral read/write layer.

Demo: Investigating Container layers

```
#podman image tree sleeper          -      sleeper is a image name in this command
```

```
#podman image tree sleeper2
```

```
#podman run -d sleeper2
```

```
#podman exec -it sleeper2 touch /tmp/samllfile
```

```
#ls ~/.local/share/containers/storage/overlay-containers/<ID>/userdata
```

5.2 Using Persistent Host Storage

Providing Persistent Storage

- Writing modifications to a small read/writable layer that is added on top of the container filesystem layers is inefficient.
 - write operations are too slow.
 - The read/writable layer is ephemeral.
- Podman can directly access persistent host storage using volumes and bind mounts.
 - Volumes are Podman objects that can be used by containers.
 - Bind mounts are data mounts managed by users.
- When accessing host storage, the container UID must have sufficient permission!

Using Bind Mounts

- Either the **-v (--volume)** option, or the **--mount** option can be used to mount persistent data.

- `#mkdir /web`
- `#podman run -p 8080:8080 --volume /web:/var/www/html:ro registry.access.redhat.com/ubi8/httpd-24:latest`

Using Volumes

- Volumes are independent objects managed by Podman.
- Use **#podman volume create**
- to create a volume, where persistent storage space is automatically allocated.
- They can refer to local host-based storage, or NFS shared storage.
- As volumes are independently managed, they can be shared between different containers.
- While creating a volume, no additional arguments are needed.

Using tmpfs Mounts

- In cases where ephemeral data is needed by an application, but for performance reasons shouldn't be stored in the container CoW filesystem, a tmpfs mount can be used.
- This uses the tmpfs filesystem on the host and mounts it to the container temporary storage.
 - `#podman run --name=tmpfs --mount type=tmpfs,tmpfs-size=1G,destination=/tmp registry.redhat.io/ubi9 dd if=/dev/zero of=/tmp/bigfile bs=1M count=1024`
 - `#podman logs tmpfile`

Importing and Exporting Data

- The **podman volume import** command can be used to import data from a tar archive into a volume.
- Use **podman volume export --output** to export data from a volume into a tar archive.

Demo: Managing Volumes

- `#podman volume create webdata`
- `#podman volume inspect webdata`
- `#echo "hello word" > index.html`

- `#cp index.html /home/student/.local/share/containers/storage/volumes/webdata/_data`
- `#podman run -d -p 8083:8080 --volume webdata:/app:Z docker.io/bitnami/nginx`
- `Curl localhost:8083`

5.3 Managing Permissions

- When a container is started, it is started in a new namespace with its own UID.
- To have this working correctly, the container UID must have permissions in that namespace.
- To monitor these permissions, use **podman unshare**:
 - `#podman unshare ls -ld /web`
- You may have to use **podman unshare** to grant permissions to the container UID before the container is started to ensure that it can access the directory.

Demo: Looking at an example

- `#mkdir /home/student/dbfiles`
- `#podman run -d -v /home/student/dbfiles:/var/lib/mysql:Z -e MYSQL_ROOT_PASSWORD=password registry.access.redhat.com/rhsc1/mysql-80-rhel7`
- `#ls -ld /home/student/dbfiles`
- `#podman inspect registry.access.redhat.com/rhsc1/mysql-80-rhel7 | grep User`

Understanding Namespaces

- Namespaces provide isolation to different aspects of computing: files, process, users, and more.
- All non-root containers are started in the same user namespace.
- User IDs exist inside that namespace only, and for access outside of the namespace UID mappings as discussed before need to be made.
- In the previous example demo, the non-root container starts a mysql process as UID 27 inside the container namespace.

- The /home/student/dbfiles directory is not a part of the container namespace, and from the container namespace, its ownership is set to root: root.

Using podman unshare to Troubleshoot

- Use **podman unshare** to verify what stuff looks like from the podman ns:
 - **Podman unshare ls -ld /home/student/dbfiles**
- Use **podman unshare** to set correct ownership, and notice that from the host OS you'll see the mapped user account.
 - **#podman unshare chown 27:27 -R dbfiles**
 - **#podman unshare ls -ld /home/student/dbfiles**
 - **#ls -ld /home/student/dbfiles**
 - **#podman top -l user huser**

5.4 Managing SELinux Settings

Managing SELinux:

- While accessing host filesystem, SELinux access control is involved.
- Containers, by default, are started with the **container_t** context type, directories mounted by containers need to be set to container_file_t.
- To ensure this happens automatically, use the **:Z** argument in the **-v** option to perform the mount, Example: **-v /web:/var/www/html:Z**
- This will automatically set the SELinux context label correctly and will also create a separate category label for each container.
- SELinux categories ensure that containers can only access their own storage.
- When using Podman volumes, SELinux configuration is handled automatically.
- **#ls -Z /dbfiles** - To see SELinux context.

5.5 Configuring Storage for Stateful Applications:

Understanding Stateful Applications

- Web-based applications are normally stateless, as they don't need to maintain state.
- Databases are typically stateful and need to persist data.
- Stateful applications have some challenges when started as a container
 - When the application is moved to a different environment, the data must be available in the new environment as well.

- You can't normally run multiple database instances writing to the same data without them synchronizing what they're doing.
- When running as a container, stateful containers must have access to the same data no matter where they are running.

Providing Configuration for Stateful Applications

- Mount persistent data on volumes and not on bind mounts.
- Use the VOLUME instruction in Containerfile to create these volumes.
- Create a dedicated network for communication between client and server processes.

Demo: Running a Database

```
#podman volume create mysqldata
```

```
#podman volume list
```

```
#podman network create datanet
```

```
#podman network list
```

```
#podman run -d --name persistent-mysql -e
MARIADB_ROOT_PASSWORD=password -v mysqldata:/var/lib/mysql --network=datanet
docker.io/bitnami/mariadb
```

Lesson 5 Lab: Configuring Container Storage

- Create a volume with the name **microservice**.
- Create a network with the name **microservice**.
- Start the bitnami/mariadb images as a container with name "backend" Ensure it mounts the "microservice" volume on the directory /data and it connects to the network "microservice".
- Start the bitnami/nginx image as a container with the name "fronted". Ensure it mounts the "microservice" volume on the directory /app and it connects to the network "microservice" Also, it should be accessible through port forwarding on localhost:8080.
- On localhost, create the file /tmp/index.html with the content "hello" from the microservice" and copy this file into the container /app directory.
- Verify it works.

Lesson 6: Troubleshooting Containers

Learning Objectives:

- 6.1 Container Logging
- 6.2 Connecting to Running Containers
- 6.3 Troubleshooting Network Issues

6.1 Container Logging

- To verify what happened while starting a container, use **podman ps -a**
- Container applications write their output and errors to the container logs.
- The container logs are stored in the read/writable layer that is added while running the container.
- For any running or stopped container, use **podman logs** to access its logs.

Demo: Troubleshooting Containers

```
#podman run --name faildb -d docker.io/bitnami/mariadb
```

```
#podman ps -a
```

```
#podman logs faildb
```

```
#podman rm faildb
```

```
#podman run --name faildb -d -e MARIADB_ROOT_PASSWORD=password  
docker.io/bitnami/mariadb
```

```
#podman ps
```

Networking Issues

- Containers are accessed by mapping a port on the host to the port used by the container application.
- If accessing the host port does not work, find out the port on which the application is running.
- To do so, use **podman exec -it CONTAINER ss -pant**
- Notice that not all containers expose a port externally, some containers connect an internal network only.

6.2 Connecting to Running Containers

- Use **podman exec** to connect to running containers.
- This allows you to run commands included in the container image.

- For commands not included in the container image, use **nsenter**, which allows you to run commands from the host within the container namespace.

Running Host Commands Within Containers

- Containers may lack many commands.
- If commands are not available, you can use **nsenter** to run a host command within the container namespace.
- To do so, you need find the container PID: **podman inspect CONTAINER-format '{{.State.Pid}}'**
- Next, use **nsenter -n -t CONTAINER_PID ss --pant** to run a host command inside the container.

Demo: Finding the Container Network Port

```
#podman run -d -p 8080:80 --name bitweb docker.io/bitnami/nginx
```

```
#podman port bitweb
```

```
#curl localhost:8080 #fails
```

```
#podman exec -it bitweb ss -pant #fails
```

```
#export BITWEBPORT=$(podman inspect bitweb --format '{{.State.Pid}}')
```

```
#nsenter -n -t $BITWEBPORT ss --pant
```

6.3 Troubleshooting Network Issues

- Containers can be connected to a network using the **--net networkname** option
- To show which networks containers want to use, type **podman inspect containername --format '{{.NetworkSettings.Networks}}'**
- Start a container on a specific network, using **podman run ... --net thisnet**
- Connect a container to specific network using **podman network connect networkname containername**

Lesson 6 Lab:

- Start the bitnami/nginx container.
- After starting it, run the appropriate commands inside the container to show a list of currently running processes as well as ports used by these applications.

Lesson 7: Integrating Podman Containers with Systemd.

Learning Objectives

7.1 Systemd User Units

7.2 Generating Systemd User Units

7.3 Automatically Updating Systemd Managed Containers

Lab: Starting Containers with Systemd

7.1 Systemd User Units

Why Systemd?

- Podman uses the fork/exec model to run containers as Linux commands.
- Anything that runs as a Linux command can be managed by the Linux default service manager Systemd.
- Systemd starts daemon processes through service unit files.
- Podman containers can be started as user processes or daemon processes using Systemd.

Auto-restarting Units

- Containers can be automatically started by Podman.
- While starting the container, use **--restart=always** to ensure the container is automatically restarted if anything happens to it.
- To have containers with this policy started in system boot, make sure you run the required Systemd services:
 - `#systemctl enable --now podman-restart`
 - `#systemctl --user enable --now podman-restart`

Creating Systemd User Units

- Any user command can be started as a user unit file.
- User unit files need to be stored in `~/.config/systemd/user`
- To automatically start a user unit file, **systemctl enable --user** can be used.
- To automatically start a user unit file without requiring the user to log in, the `linger` feature must be enabled for that user.
- Use **loginctl enable-linger username** to do so.

Demo: Starting a Systemd User Unit

```
#useradd anna
```

```
#passwd anna
```

```
#loginctl enable-linger anna
```

```
#ssh anna@localhost
```

```
#mkdir -p ~/.config/systemd/user
```

```
#cd ~/.config/systemd/user - To go inside this directory.
```

```
#vim sleep.service
```

```
[Unit]
```

```
Description=Sleep Service
```

```
[Service]
```

```
Type=simple
```

```
ExecStart=/bin/sleep infinity
```

```
[Install]
```

```
WantedBy=default.target
```

```
:wq! - To save and quit
```

```
#systemctl --user daemon-reload - To reload daemon
```

```
#systemctl --user enable --now sleep.service
```

```
#systemctl --user status sleep.service
```

7.2 Generating Systemd User Units

Generating Systemd User Units:

- The **podman** command can generate user unit files for running containers.
- To do so, the actual user should be in the directory `~/.config/systemd/user`
- Next, use **#podman generate systemd --name containername -n --new --files**
 - Use **--files** to generate a service file.
 - Use **-n** to create the service file by name and not container ID.
 - Add **--new** to stop the currently running container and create a new one.

Demo: Generating a Systemd Container Unit

- Run the commands as user anna created in the previous demo

```
#cd ~/.config/systemd/user
```

```
#podman run -d -p 8082:8080 --name annademo docker.io/bitnami/nginx
```

```
#podman generate systemd annademo --files -n
```

```
#systemctl --user daemon-reload
```

```
#systemctl --user status container-annademo
```

```
#systemctl --user enable container-annademo
```

```
#su root
```

- As non-anna user

```
#ps faux | grep -C5 nginx
```

7.3 Automatically Updating Systemd Managed Containers

Managing Container Auto-updates

- When Podman containers are managed by systemd, they can be updated automatically.
- To do so, the container must have a label set to `io.containers.autoupdate=registry`.
- Use **#systemctl enable --now podman-auto-update.timer** to run a daily automatic update.
- Alternatively, use **#podman auto-update** to trigger this process manually.
- Use **#podman auto-update --rollback** to roll back to the previous version if the update has failed.

Demo: Managing Container Auto-updates

```
#podman run --label "io.containers.autoupdate=registry" -p 8083:8080 --name demo1  
docker.io/bitnami/nginx
```

```
#cd ~/.config/systemd/user
```

```
#podman generate systemd -n --files --new demo1
```

```
#podman stop demo1
```

```
#systemctl --user enable --now container-demo1
```

```
#podman auto-update
```

```
#systemctl enable --now podman-auto-update.timer
```

Lesson 7 Lab: Starting Containers with Systemd

- Create a file index.html with the contents “version 1”.
- Create a Containerfile that uses the docker.io/bitnami/nginx image and copies the index.html into the container /apps directory.
- Build an image with the name lab7 based on this containerfile.
- Run a local registry Pod on localhost port 5000.
- Push the lab7 image to the localhost registry.
- Run the container lab7 from this image and ensure it is prepared for automatic updates. It should be accessible through localhost port 8088.
- Generate Systemd unit file that manages this container.
- Stop the currently running container and start and enable it through the systemd unit file.
- Verify the localhost port 8088 prints “version1”.
- Enable and start the systemd podman-auto-update.timer unit.
- Change the contents of the index.html file to read “version2”.
- Update the image and upload the updated image to the image registry running at localhost.
- Manually run a Podman auto-update.
- Verify that the automatic update was performed successfully and “version2” is printed on localhost port 8088.

```
#vim index.html - To create a index file.
```

```
Version1
```

```
:wq! - To save and quit.
```

```
#mkdir lab7
```

```
#mv index.html lab7/
```

```
#cd lab7
```

```
#ls
```

#vim Containerfile - To create a Containerfile like Dockerfile

```
FROM docker.io/bitnami/nginx
WORKDIR /apps
COPY index.html .
```

:wq! - To save and quit

#podman build -t localhost:5000/lab7 . - To build a Image.

#podman push localhost:5000/lab7 - To push localhost

#podman auto-update

Lesson8: Microservices and Multi-container Applications

Learning Objectives:

8.1 Understanding Microservices

8.2 Using Compose to Run and Manage Microservices

8.1 Understanding Microservices:

What are Microservices?

- A microservices in an application that consists of different independent components that are connect together.
- The independent components can be individually developed by completely disconnected projects.
- An additional layer is used that adds site-specific information to the microservices.

Why use Microservices?

- Developing independent, small applications is easier than maintaining one big application.
- Changes can be applied faster.
- By applying common DevOps technologies like Continuous Integration /Continuous Development, zero-downtime application upgrades can be provided.

Microservices Benefits

- When broken down in pieces, applications are easier to build and maintain.
- Each piece is independently developed, and the application itself is just the sum of all pieces.
- Smaller pieces of application code are easier to understand.

- Developers can work on applications independently.
- If one component fails, you spin up another while the rest of the application continues to function.
- Smaller components are easier to scale.
- Individual components are easier to fit into continuous delivery pipelines and complex deployment scenarios.

Understanding Containers and Microservices

- A container is an application that runs based on a container image. The container image is a lightweight, standalone, executable package of software that includes everything that is needed to run an application.
- Because containers are lightweight and include all dependencies required to run an application, containers have become the standard for developing applications.
- As containers are focusing on their specific functionality, they are perfect building blocks for building microservices.
- In a containerized, microservices environment, site-specific information can be provided in a flexible way by using orchestration tools such as Kubernetes.

8.2 Using Compose to Run and Manage Microservices

Using Microservices in Podman

- **podman-compose** uses a YAML file `compose.yaml` that defines one or more containers with their required properties.
- This allows you to run connected containers in a development environment.
- **Podman-compose** is not the recommended way to run connected containers in production: better use Kubernetes or OpenShift.
- As an alternative, Podman can use Pods, which are based on the way containers are started through Kubernetes YAML files.
- Use **podman generate kube** to generate these YAML files from existing containers.
- Use **podman play kube** to create containers based on Kubernetes YAML files.

Using podman-compose

- To use **podman-compose**, you'll first to install it.
 - **#subscription-manager repos --enable codeready-builder-for-rhel9-x86_64-rpms**
 - **dnf install <https://dl.fedoraproject.org/pub/epel/epel-release-latest-9.noarch.rpm>**
 - **#dnf upgrade**

- **#dnf install podman-compose**
- Write a compose.yaml file with the instructions you want to execute.
- Use **podman-compose up** to process the instructions in the compose file and start all services specified.
- To start all containers in detached mode, use **podman-compose up -d**

Understanding Compose.yaml:

- In a compose.yaml file, different resources are created
 - services: these are the containers with all of their parameters. Multiple services can be created as child elements to the services parameter.
 - volume: allows for automated creation of storage volumes.
 - networks: allows for the creation of dedicated networks.
- To specify container properties, use the following:
 - Image: the complete reference to the image you want to use.
 - Networks: the networks to which you want the container to connect.
 - Ports: port mappings (equivalent to the **podman run -p** option).
 - Environment: environment variables you'd like to set.

Lesson 8 Lab: Using Compose

- Create Compose file that brings up a microservices according to the following requirements:
 - Defines a volume datavol
 - Defines a network datanet
 - Defines a container named frontend, using docker.io/nginx:latest, which mounts the datavol on /app, connects to the datanet network and is accessible using hostport 8008 that forwards to container port 8080
 - Defines a container backend, using docker.io/bitnami/mariadb:latest, which connects to the datanet network and uses the environment variable MARIADB_ROOT_PASSWORD set to password, as well as MARIADB_DATABASE set to the value databooks.
 - Ensure all containers are started in detached mode.

Lesson 9: An Introduction to Container Orchestration

9.1 Kubernetes and OpenShift

9.2 Taking an OpenShift Test-drive

9.1 Kubernetes and OpenShift:

- OCI standardized container images can be used on your local computer, but also in cloud.
- For running images in cloud, an orchestration platform is needed.
- Orchestration takes care of automated managing, scaling and networking of containers in cloud and datacenter environments.
- OpenShift is the Red Hat Kubernetes distribution.

Understanding Orchestration:

- Applications running in clouds should run in a decoupled way.
- That means they shouldn't have a direct relation with any specific server.
- To run decoupled applications successfully, specific resources are needed to store and manage application-related information.
- Containers are started in Pods. The Pod adds cloud properties to the container.
- To provide for scalability, Pods are started through Deployment.
- Configuration is not provided through files, but using ConfigMaps or base64 encoded Secrets.
- To provide access to applications, services are used.

OpenShift Features:

- OpenShift is the Red Hat Kubernetes distribution, which comes with lots of developer-oriented features.
- These developer features allow for easy integration of CI/CD through technologies like Source to images.
- This allows you to run applications directly from code, where after an update to the source code your application will be updated as well.
- To deal with the many features that OpenShift has added to Kubernetes, the `oc` client is provided.

9.2 Taking an OpenShift Test-drive

Using OpenShift

- A fully installed OpenShift environment involves many resources.
- OpenShift Local is an environment that can be installed on Windows, Mac, or Linux.
- You'll need 12 GiB RAM to run OpenShift Local.
- The OpenShift Sandbox is a 30-day, limited, cloud-based environment that is available for free on developers.redhat.com.

Demo: Running an Application in OpenShift

- In the OpenShift developer sandbox, start a command line terminal (this will take a few minutes).

- Type **oc run --image=bitnami/nginx** to start a container based on the bitnami/nginx image
- Use **oc get all** for an overview of running OpenShift resources.
- Type **oc create deploy webshop --image=bitnami/nginx --replicas=3** to start a replicated application.
- Use **oc get all** to verify that it is running.

Lab: Running Applications in OpenShift

- Start OpenShift Developer Sandbox.
- Run a replicated application with the name webapp based on the bitnami/nginx image and ensure this application has 3 running instances.

Lesson 10: Sample Exam

Learning Objectives

10.1 Running a Container

10.2 Building a Custom Image

10.3. Running a Microservice

10.4. Analyzing Running Container

10.5 Running a Local Registry

10.6 Creating Custom Image from Running Container

10.7 Managing Container Users and Permissions

10.1 Running a Container

- Run all tasks as restricted user “student” unless stated otherwise.
- Run the bitnami.mariadb container image as a rootless container with the name booksdb and make sure you meet the following requirements.
 - A Podman volume with the name mariavol need be created on the host. It should be mounted within the container on the directory /var/lib/mysql.
 - The container should be accessible through host port 3206.
 - While starting, the variable MARIADB_ROOT_PASSWORD must be set to the value “password”.
 - The container should be restricted to a maximum usage of 128MiB.
 - Make sure this container is automatically started by systemd, also when user student isn’t logged in.

10.2 Building a Custom image

- Create a custom image with the name “task2” that meets the following requirements

- The image uses bitnami/mariadb as the base image.
- The image defined the argument DB_PASSWORD.
- The value of this argument is set to “task2db” and is provided while building the image.
- The argument is used in the environment variable MARIADB_ROOT_PASSWORD that should be hard coded in the image.
- Run the container task2 based on this image, where the default container port 3306 is accessible through host port 3307.

10.3 Running a Microservices

- Use **podman-compose** to create a microservices that meets the following requirements
 - A network the name micronet is created. All containers connect to this network.
 - A volume with the name microvolt is created.
 - A container with the name frontend is used to start the docker.io/bitnami/nginx image. Its default port 8080 is available on host port 8181.
 - A container with the name backend is used to start the docker.io/bitnami/mariadb image. The MARIADB_ROOT_PASSWORD variable is set the value password.
 - All containers in the microservices are started in detached mode.

10.4 Analyzing Running Containers

- Run the container “task4”, based on the registry.redhat.io/ubi9/ubi image. Make sure available memory is limited to 1GiB and run the container in detached mode.
- From the host operating system, use the **free** command to verify available memory in the container.

Sample Exam:

Running a Local Registry

- Start a container with the name “registry”, which is based on the docker.io/library/registry image.
- Ensure this container is accessible through host port 5000.
- Ensure this container is automatically started by Systemd.
- Upload the “task2” image created earlier to this registry.

Creating a Custom Image from a Running Container

- Run a container based on the docker.io/library/nginx image.
- In this container, create a file with name /tmp/datafile, containing the text “my data”.

- Create a new image with the name customnginx, that is based on this image, and which includes the custom file you've created.

Managing Container Users and Permissions

- Create a custom container image with the name touchfile, based on the ubi9/ubi image. The image should run as the user fileman, with UID 1111. As the command, the container image should create an empty file /data/newfile, using the Linux command **touch**.
- Run a container with the name filecreate, that bind-mounts the directory ~/data in the container /data directory, such that while running the container the /data/newfile is created in a persistent way in the ~/data directory.

